

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1707

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Duration : 45 Minutes

QUESTIONS: 2

Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I P.Kalyani(192472199) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Kalyani

ANNEXURE A

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

Decision Tree Classification Workflow

Cleaning → splitting → training → prediction → evaluation

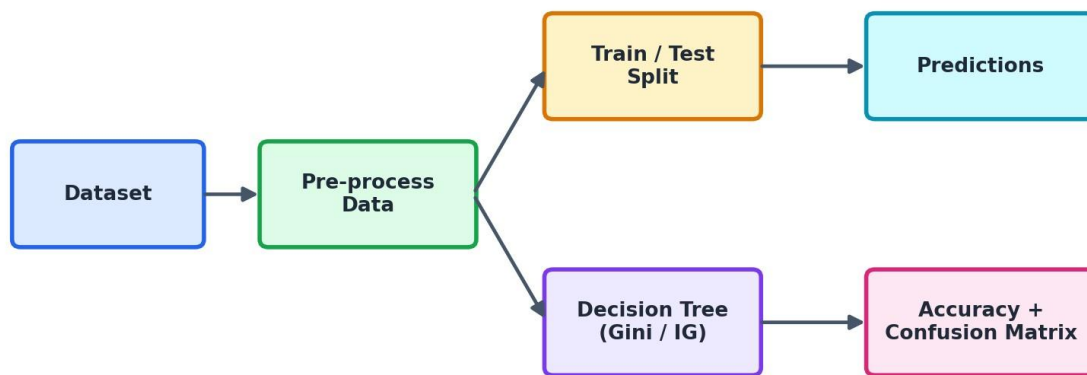


Figure 1: Experimental workflow

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

Answer: A Decision Tree classification experiment begins by loading the selected dataset, such as Iris or Play-Tennis. The data must be inspected before model training so that missing values, categorical variables and incorrect data types do not affect the classifier.

The first step is to load the dataset into a DataFrame. The first five records are displayed using the equivalent of `head(5)`, while data types are inspected to determine which columns are numerical and which are categorical. Missing numerical values can be handled using an appropriate statistic such as the mean or median, while categorical missing values can be

replaced with the most frequent category. If categorical attributes are present, they can be encoded into numerical form using label encoding or one-hot encoding as appropriate.

After preprocessing, the independent features X and target variable y are separated. The dataset is then divided into training and testing sets. A common split is 80% for training and 20% for testing. The training set is used to learn the decision rules, while the testing set is kept separate for evaluating how well the model generalizes to unseen samples.

Illustrative Python implementation:

```
import pandas as pd
from sklearn.model_selection import
train_test_split
data = pd.read_csv("dataset.csv")
print("First 5 rows:") print(data.head())
print("\nData types:") print(data.dtypes)
print("\nMissing values:")
print(data.isnull().sum())
# Example: fill numerical missing values
num_cols = data.select_dtypes(include=["int64",
"float64"]).columns
for col in num_cols:
    data[col] = data[col].fillna(data[col].median())
# Example target column
X = data.drop("target", axis=1)
y = data["target"]
# Encode categorical features when required
X = pd.get_dummies(X, drop_first=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
```

```
print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

This preprocessing stage is important because the quality of the input data directly affects the quality of the learned decision tree. The train-test split also prevents the evaluation from being performed on the same observations used to train the model.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification.

Answer: A Decision Tree recursively divides the training data into smaller groups by selecting features and split points that produce the most useful separation of classes. The split can be selected using Gini Index or Information Gain.

For the Gini criterion, impurity is calculated as:

$$\text{Gini} = 1 - \sum(p_i^2)$$

where p_i is the proportion of samples belonging to class i in a node. A lower Gini value indicates a purer node.

For Information Gain, the reduction in entropy produced by a split is measured. The feature that provides the greatest useful reduction in impurity is preferred as the split.

Illustrative implementation using Gini:

```
from sklearn.tree import DecisionTreeClassifier,
plot_tree import matplotlib.pyplot as plt model =
DecisionTreeClassifier(      criterion="gini",
random_state=42
)
model.fit(X_train, y_train)
print("Decision Tree trained
successfully.") plt.figure(figsize=(14, 8))
plot_tree(      model,
```

```

        feature_names=X.columns,
        class_names=[str(c) for c in model.classes_],
        filled=True,        rounded=True
    )
plt.title("Decision Tree Structure") plt.show()

```

The root node is the first decision node at the top of the tree. It is selected because the chosen splitting criterion identifies that feature and threshold as the best initial division of the training data. Therefore, the root node is justified by the maximum impurity reduction, or equivalently the best split according to the selected criterion. The exact root feature depends on the dataset used in the experiment, so it should be reported from the generated tree rather than assumed in advance.

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Answer: After training, the Decision Tree is evaluated using the test data. Accuracy measures the proportion of correctly classified test samples. The confusion matrix shows how actual classes are distributed against predicted classes, making it possible to identify the classes that are confused by the model. The classification report provides precision, recall and F1-score for each class.

Illustrative implementation:

```

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred) cm
= confusion_matrix(y_test, y_pred)

```

```
print("Accuracy:", accuracy)
print("\nConfusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
# Identify misclassified instances
misclassified = X_test[y_test.values != y_pred]
print("\nMisclassified instances:")
print(misclassified.head(10))
```

Performance should be interpreted using all three measures rather than accuracy alone. A high accuracy indicates that most test observations were classified correctly, while the confusion matrix reveals whether errors are concentrated in a particular class. Precision indicates how many predicted instances of a class were actually correct, recall indicates how many actual instances of that class were detected, and F1-score balances precision and recall.

At least two misclassified instances should be listed from the actual output of the experiment. Their actual labels and predicted labels should be compared to explain the errors. Since the uploaded assessment specifies Iris / Play-Tennis as examples but does not provide one fixed dataset or the resulting test predictions, the exact two misclassified records cannot be truthfully named without running the experiment on the selected dataset.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

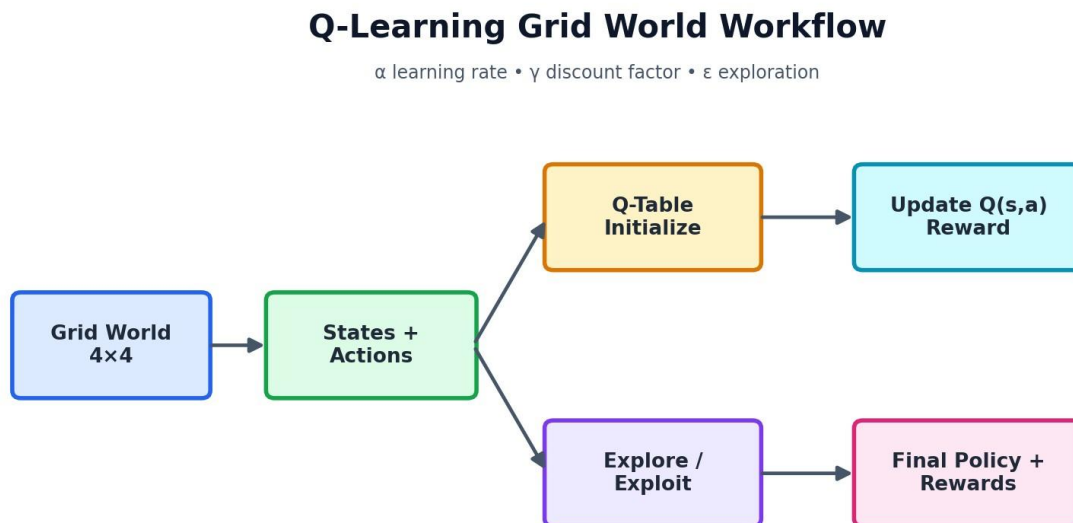


Figure 2: Experimental workflow

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).

Answer: The Grid World is represented as a 4×4 environment containing 16 possible states. Each cell can be represented using a state index from 0 to 15. The agent can select four actions: Up, Down, Left and Right. The environment returns a reward according to the outcome of the selected action.

The reward structure specified in the question is:

- +10 when the agent reaches the goal.
- -1 for every normal step.
- -5 when the agent encounters an obstacle.

The Q-table stores the expected long-term value of taking an action in a particular state. Since there are 16 states and 4 actions, the initial table contains 16×4 entries. All entries are initialized to zero because the agent has no prior knowledge of which actions are useful.

The major hyperparameters are:

- α (alpha): learning rate, controlling how strongly new information changes the existing Qvalue.
- γ (gamma): discount factor, controlling the importance of future rewards.
- ϵ (epsilon): exploration probability in an epsilon-greedy policy.

A suitable starting configuration is $\alpha = 0.1$, $\gamma = 0.9$ and $\epsilon = 0.1$. These values are example experimental settings and can be tuned during evaluation.

Illustrative initialization:

```
import numpy as np
num_states = 16 num_actions
= 4
# Actions: 0=Up, 1=Down, 2=Left, 3=Right
q_table = np.zeros((num_states,
num_actions)) alpha = 0.1 gamma = 0.9 epsilon
= 0.1
print("Q-table shape:", q_table.shape)
print("Initial Q-table:")
print(q_table)
```

The environment should also define the goal state, obstacle states, boundary behavior and transition rules. These rules determine the next state and reward after every action.

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

Answer: Q-Learning learns through repeated interaction with the Grid World. During every episode, the agent starts from a defined initial state and repeatedly selects an action using an

epsilon-greedy strategy. The selected action produces a next state and reward, after which the Q-value is updated.

The Q-Learning update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where s is the current state, a is the selected action, r is the received reward, and s' is the next state.

A suitable implementation is:

```
for episode in range(1, 101):
    state =
start_state      done =
False          while not
done:
    if np.random.random() < epsilon:
        action = np.random.randint(num_actions)
    else:
        action = np.argmax(q_table[state])
    next_state, reward, done = environment_step(state,
action)
    old_value = q_table[state, action]
next_best = np.max(q_table[next_state])
q_table[state, action] = (
old_value +
        alpha * (reward + gamma * next_best -
old_value)
        )
    state = next_state    if
episode in [1, 50, 100]:
print("Episode:", episode)
    print("State 0 Q-values:", q_table[0])
print("State 5 Q-values:", q_table[5])
```

Two representative states can be selected, for example state 0 and state 5. Their Q-values should be recorded at Episodes 1, 50 and 100. Initially, values are near zero because the agent has not learned the environment. As training progresses, actions that lead toward the goal should obtain larger values, while actions that repeatedly cause negative rewards should remain lower.

The exact numerical Q-values depend on the starting state, obstacle locations, random exploration and implementation details. Therefore, the assessment output should record the actual values generated by the student's execution rather than inventing fixed numbers.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Answer: After 100 or more episodes, the learned policy is extracted by selecting the action with the maximum Q-value for each state. The resulting action indicates what the agent currently believes to be the best decision in that state.

Illustrative implementation:

```
actions = ["Up", "Down", "Left", "Right"]
for state in range(num_states):
    best_action = np.argmax(q_table[state])
print(
    "State:", state,
    "Optimal Action:", actions[best_action]
)
```

The policy can also be displayed as a 4×4 grid so that the selected action at each cell can be inspected visually. The goal state can be marked separately, while obstacle cells can be shown as blocked.

Cumulative reward should be recorded after every episode. A typical implementation is:

```
episode_rewards = []
for episode in range(1, 101):
    total_reward = 0
    # Run one complete episode here
    # total_reward is updated after every action
    episode_rewards.append(total_reward)

plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Cumulative Reward per Episode")
plt.show()
```

Convergence is indicated when the agent's performance becomes more stable over successive episodes. In a successfully learned environment, the cumulative reward generally improves from the early training stage and then fluctuates around a comparatively stable level as the agent repeatedly follows a good route toward the goal.

The exact convergence curve depends on the reward structure, obstacle placement, exploration rate, learning rate and random initialization. If epsilon remains fixed, occasional exploratory actions can continue to cause fluctuations even after a good policy has been learned. A decreasing epsilon can gradually reduce exploration and make the final policy more stable.

Therefore, the final policy demonstrates the agent's learned decision-making strategy, while the cumulative reward plot provides evidence of whether learning is improving and stabilizing over time.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation